Introduction to computational physics

Lecture 1

PHYS 246 class 1
Fall 2026
J Noronha-Hostler

https://jnoronhahostler.github.io/IntroductionToComputationalPhysics/intro.html

Why computational physics?

- Most fields of physics have problems that cannot be solved just using pen and paper and require computational algorithms
- Computational simulations can predict the behavior of experiments, allow physicists to compare models to data - leading to new insights, and handle large data sets
- Computational physicists are in high demand, especially with advances in machine learning and artificial intelligence tools. Advances in research lead to industry advances down the road
- Starting good coding practices NOW, can make your life much easier down the road.

About me

- Co-founder of the MUSES collaboration (led by UIUC)

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

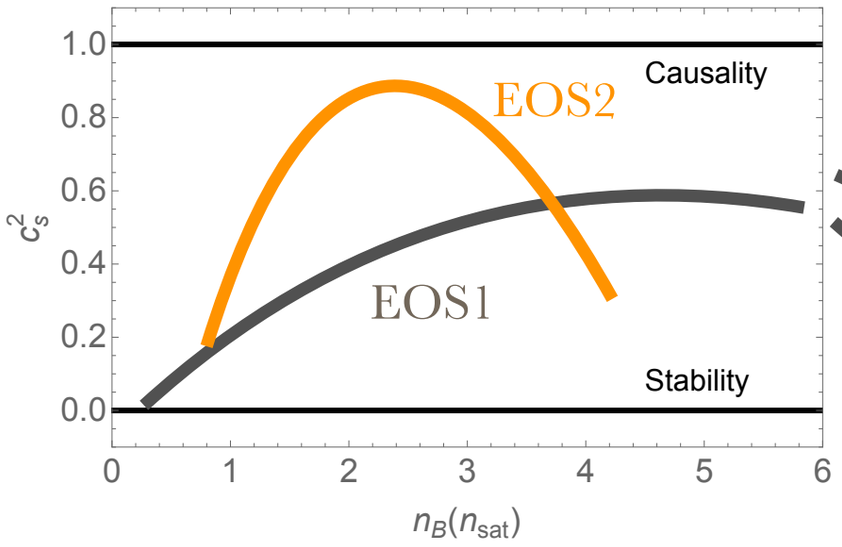
muses

- Co-developer of a number of codes:
  - https://github.com/the-nuclear-confectionery
  - https://gitlab.com/nsf-muses/module-cmf
  - https://gitlab.com/nsf-muses
- My own expertise lies in c++, Fortran, Mathematica and high-performance computing

Peering inside neutron stars

Dead stars = largest densities in the universe!

Generate $> 10^5$ equations of state (EOS), test against data



Assign a likelihood to each EOS

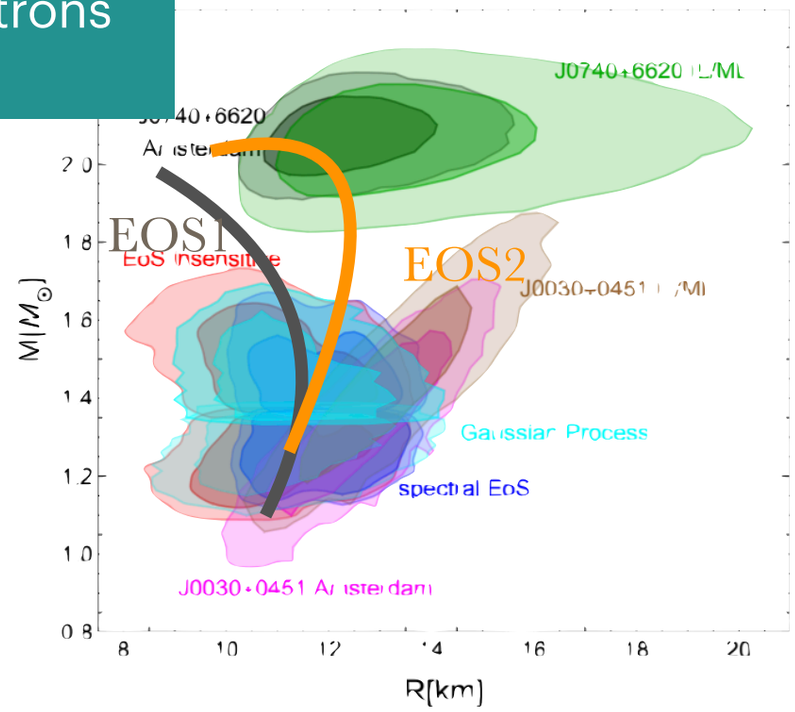
$$Posterior \propto Likelihood \times Prior$$

Independent measurements:

$$\mathscr{L} = \prod_{i=M,R,\ldots} \left[\prod_{j=1}^{j(i)} \mathscr{L}(i,j) \right]$$

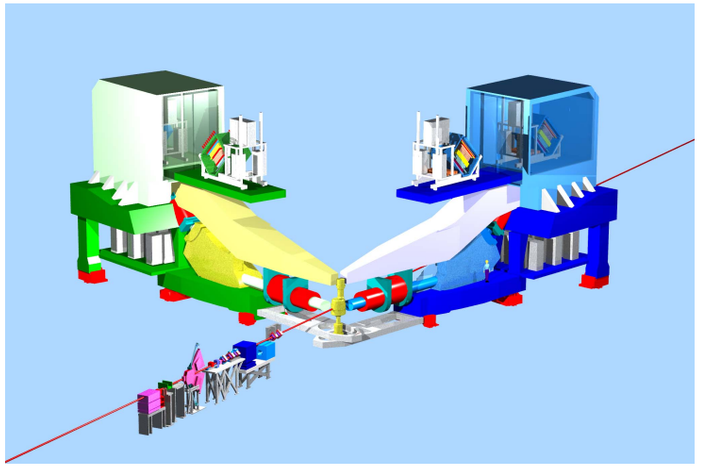
Isolated, cold neutrons stars/Inspiral

Astrophysical constraints



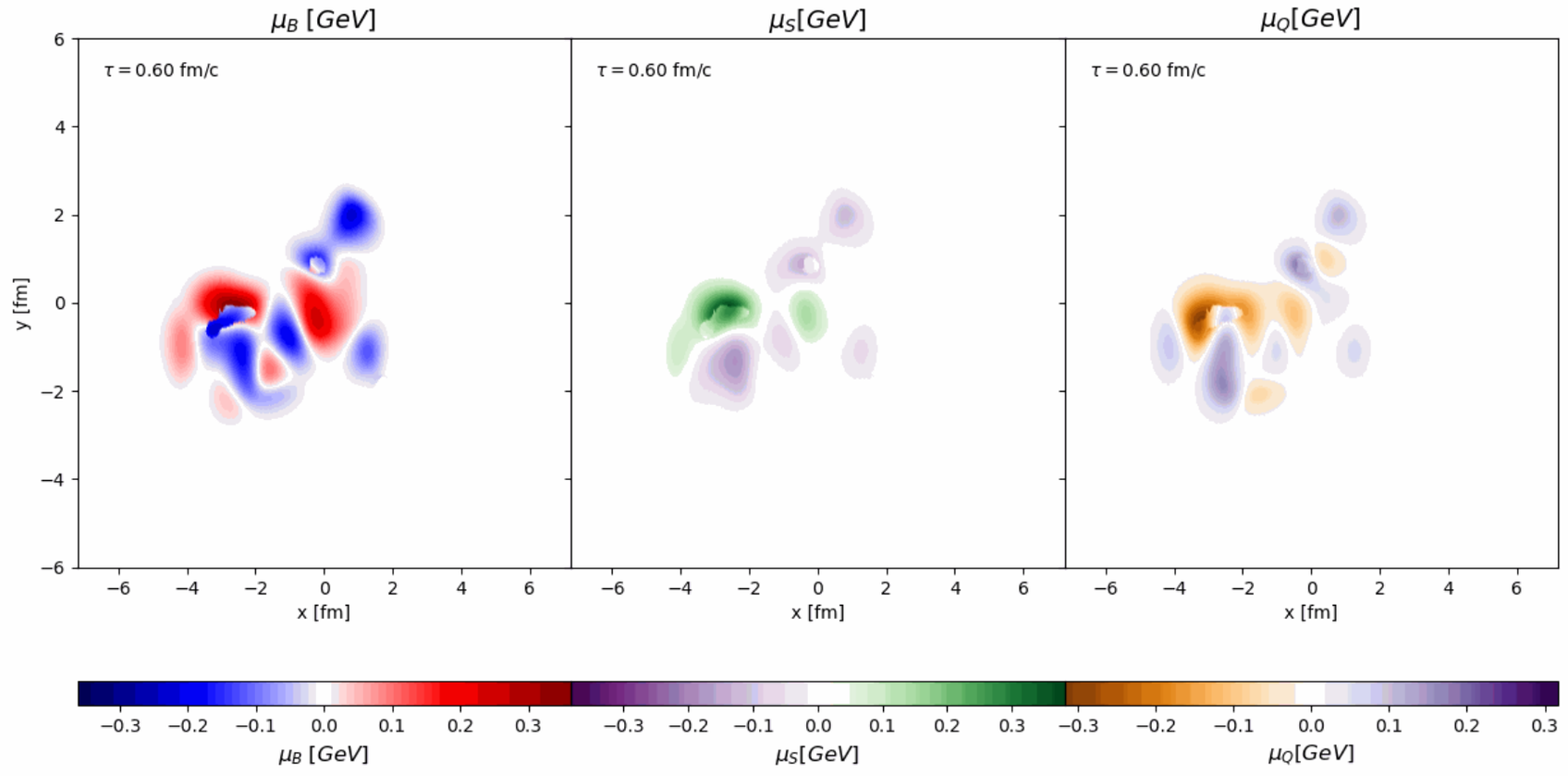
Nuclear experiments/saturation

Theory constraints



Relativistic viscous hydrodynamics

Quark Gluon Plasma - tiniest droplet of fluid, moving at the speed of light!



$$D\varepsilon = -(\varepsilon + P)\theta - \Pi\theta + \pi_{\mu\nu}\sigma^{\mu\nu}$$

$$(\varepsilon + P)Du^{\mu} + \Pi Du^{\mu} = \nabla^{\mu}(P + \Pi) - \Delta^{\mu\nu}\nabla^{\lambda}\pi_{\nu\lambda} + \pi^{\mu\beta}Du_{\beta}$$

$$\tau_{\pi}\dot{\pi}^{\mu\nu} + \pi^{\mu\nu} = 2\eta\sigma^{\mu\nu} - \frac{\tau_{\pi}\pi^{\mu\nu}}{2\beta_{\pi}}\dot{\beta}_{\pi} - \frac{\tau_{\pi}}{2}\pi^{\mu\nu}\theta + \ldots$$

$$D\rho_q = -\rho_q\theta + n_q^{\mu}Du_{\mu} - \nabla_{\mu}n_q^{\mu}$$

$$\tau_{\Pi}\dot{\Pi} + \Pi = -\zeta\theta - \frac{\tau_{\Pi}\Pi}{2\beta_{\Pi}}\dot{\beta}_{\Pi} - \frac{\tau_{\Pi}}{2}\Pi\theta + \ldots$$

$$\tau_{qq'}\dot{n}_{q'}^{\mu} + n_q^{\mu} = -\kappa_{qq'}\nabla^{\mu}\alpha_{q'} + \frac{\tau_{qq'}n_{q'}^{\mu}}{2\beta}\theta - \frac{\tau_{qq'}}{2\beta}\dot{\beta}_{q'l}n_l^{\mu} + \ldots$$

Research opportunities

- Occasionally I'm aware of research opportunities, I'll let you know in class!
- That being said, if you already know **c++ (well)**, feel free to get in touch. There's often coding opportunities for up to a few students. Send me and email with your CV+ a few words about your past experience.

Great TAs!

Nikolas Cruz C.:
cnc6@illinois.edu

Computational nuclear astrophysics: lead developer of CMF++, a Chiral Mean Field model code for MUSES collaboration to simulate ultra-dense matter in neutron stars. Expertise in Fortran, C/C++, Python, High-Performance Computing (HPC)

Kaitlyn Prokup:
kprokup2@illinois.edu

Computational astrophysics: developer for GWAT, a Gravitational Wave Analysis Tools library written in C++ for performing statistical studies on gravitational wave data. Expertise in C/C++, Java, Python, High-Performance Computing (HPC)

Office Hours

Monday: Nikolas at 4pm

Tuesday: Jaki at 1:15pm-2pm, Loomis 427

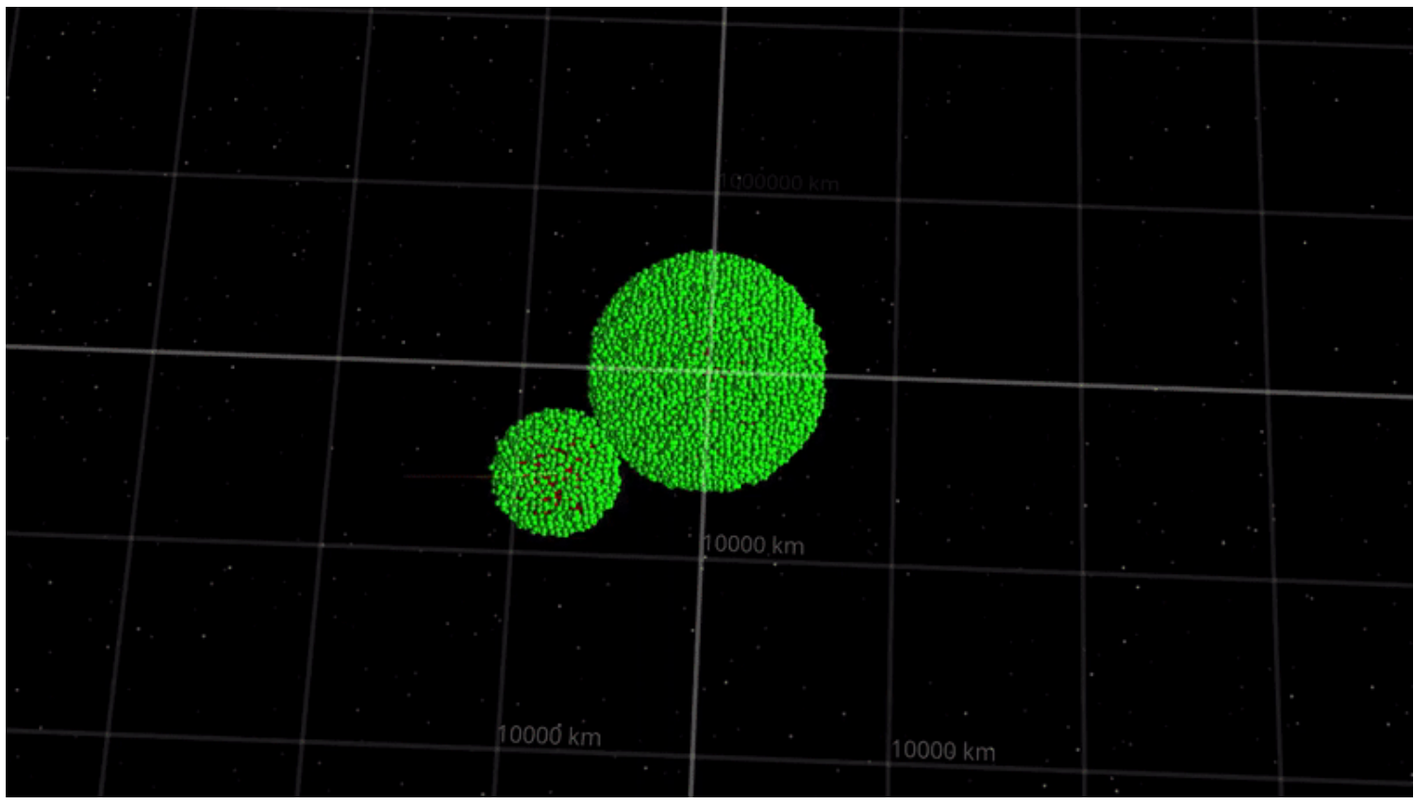
Wednesday: Kaitlyn at 3pm

Class Structure & Attendance

- First part of class is a **quiz** on the previous assignment
- Second, there's lecture/overview of the problem
- Last part of class provides free time to work on the assignment, ask questions, work in groups etc (attendance not mandatory, but encouraged)
- Why should you attend and participate?
  - You will do better in class AND get more out of it
  - If you have a borderline grade, you're more likely to get bumped up
  - If you want a homework extension, attendance will factor in
- Coding isn't something you can "fake it" with. If you get a job to code and you can't, you won't be there long. It's in your own interest to learn this material as best as you can.

Common tools

- Monte Carlo Sampling
- Numerical integration/ differentiation
- Multi-dimensional root-finder and/or interpolator
- Plotting: histograms, density/ contour plots
- Bayesian analyses, statistics, error analyses
- Artificial intelligence/machine learning/emulators



Taken from https://universesandbox.com/blog/2019/12/sph-devlog-1/

This class...

- Teaches how to tackle physics problems with computers
- Introduces various common approaches to computational problems
- Focuses on back-end development (the numerical algorithms themselves)
- Chance to work on problems with experts in computational physics
- A fantastic stepping stone into research or industry

This class is NOT...

- A class on machine learning/ artificial intelligence
- A basic CS course (I will expect a certain level of knowledge)
- A class on front-end development (e.g. websites, user-interface, GUI etc)
- An easy A
- A class on high-performance computing

Getting started

Click here to go to google collab

N ways to measure π

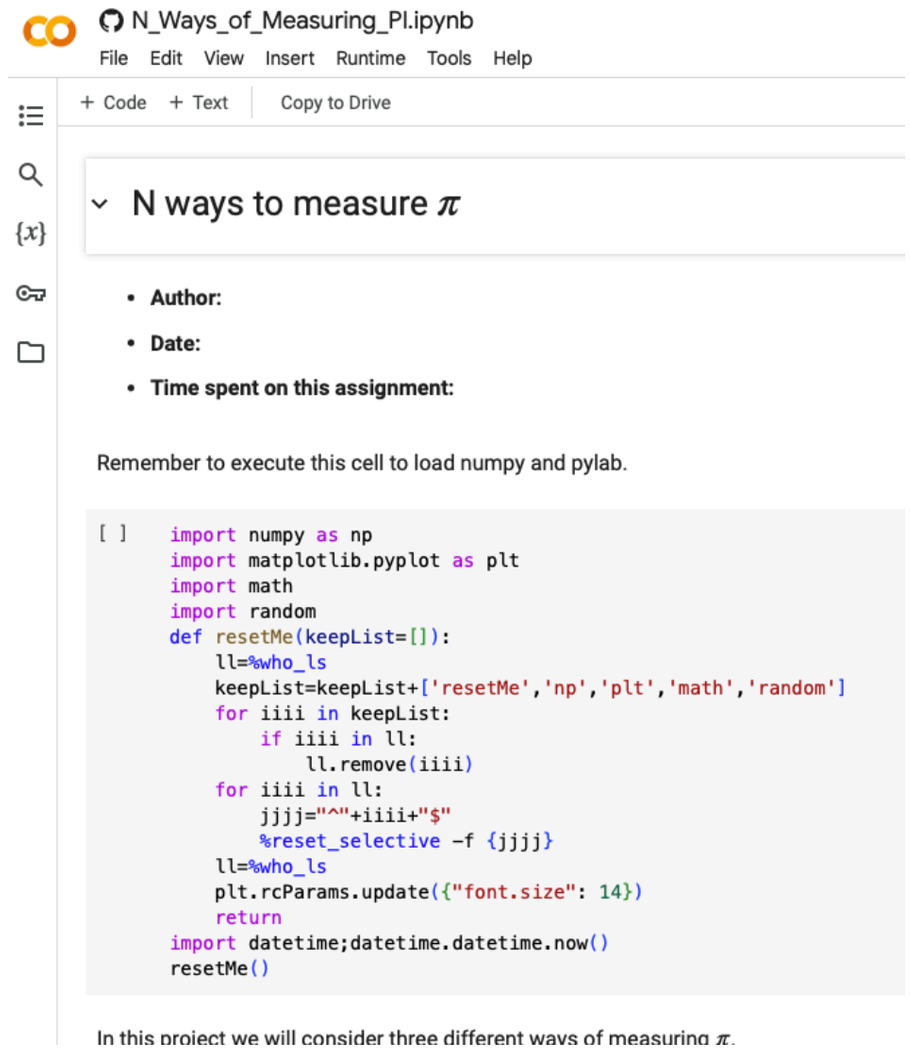
- **Author:**
- **Date:**
- **Time spent on this assignment:**

Remember to execute this cell to load numpy and pylab.

▶ Show code cell content

In this project we will consider three different ways of measuring π.

Confused about what exactly you need to do?
Look for the owl emoji 🦉 for instructions



N_Ways_of_Measuring_PI.ipynb

File Edit View Insert Runtime Tools Help

+ Code + Text Copy to Drive

N ways to measure π

- **Author:**
- **Date:**
- **Time spent on this assignment:**

Remember to execute this cell to load numpy and pylab.

```
[ ]  import numpy as np
     import matplotlib.pyplot as plt
     import math
     import random
     def resetMe(keepList=[]):
         ll=%who_ls
         keepList=keepList+['resetMe','np','plt','math','random']
         for iiii in keepList:
             if iiii in ll:
                 ll.remove(iiii)
         for iiii in ll:
             jjjj="^"+iiii+"$"
             %reset_selective -f {jjjj}
         ll=%who_ls
         plt.rcParams.update({"font.size": 14})
         return
     import datetime;datetime.datetime.now()
     resetMe()
```

In this project we will consider three different ways of measuring π.

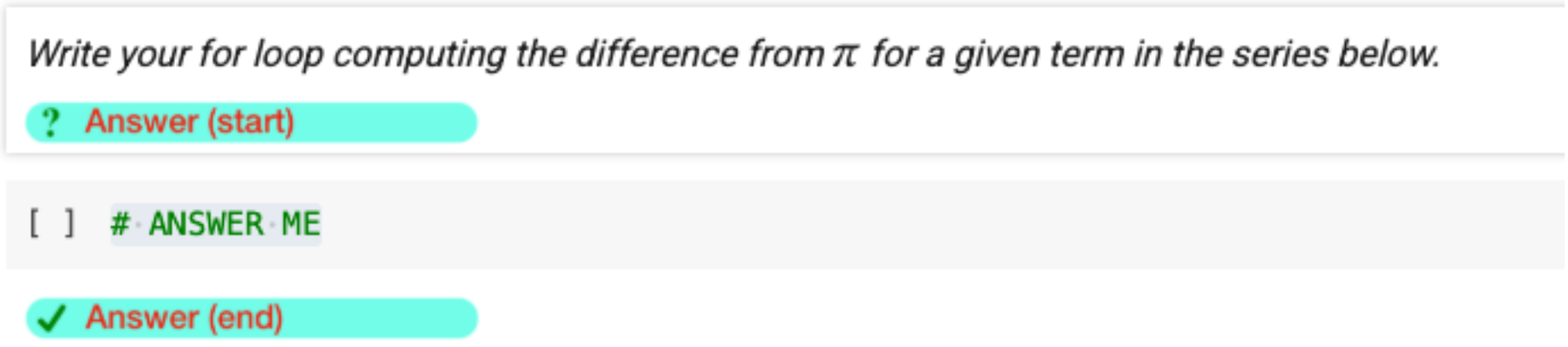
After opening, go to File -> Save a Copy in Drive and add your name to the title.

Note: if you reopen from the class website, it creates a *new* notebook, so always returned to your saved version!

Python notebooks

Runtime environment

- Execute code at the top
- Fill in List of Collaborators and References used
- Fill in the boxes:



- Large Language Models (LLMs) e.g. ChatGPT, are strongly discouraged

Good coding practices

Example notebook: https://colab.research.google.com/drive/1WeZuY1Qt9ds5vaIBAcf75VNRIuMBMjuE?usp=sharing

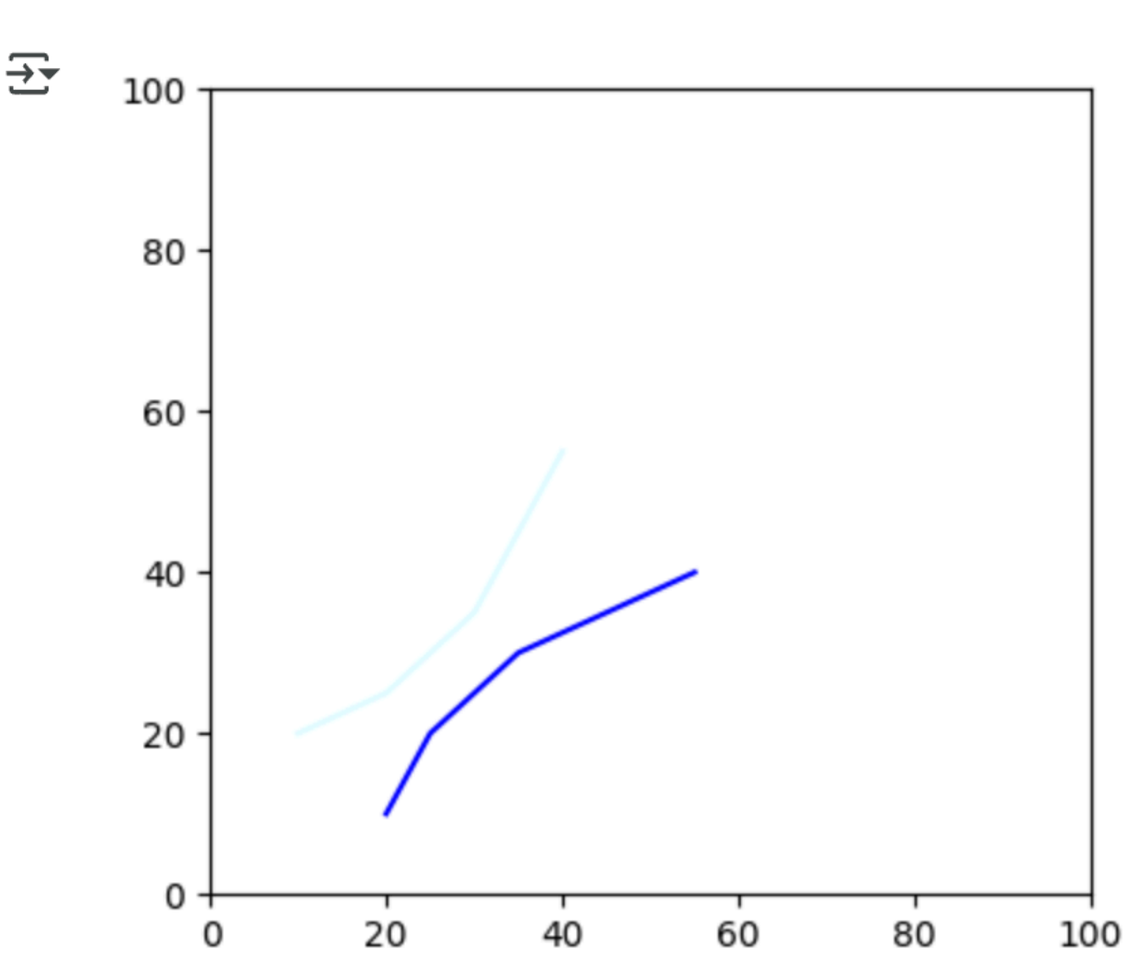
- Clear names: “Temperature” instead of “par1”
- Document code: add comments explaining steps
- Delete excess/practice code

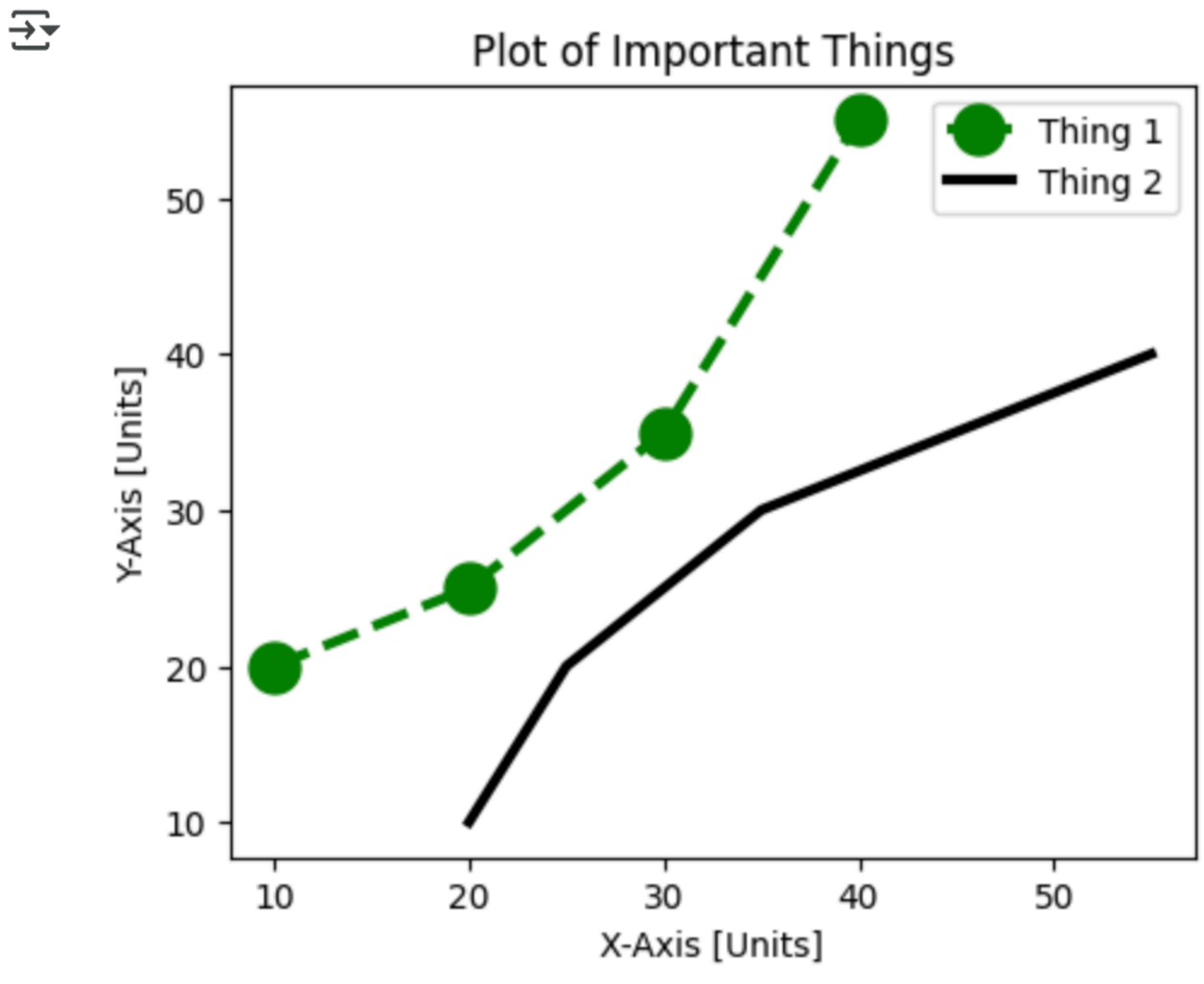
```
[2] # Let's start with variable names.
    #It's not a great idea to use things like x,y,z because it's not clear what these are and it's easy to forget if you haven't looked at your code for a long time

    #Bad code:
    x=2
    y=3.0
    z=20
    out=x*y*z
    print(out)

    #instead, say what it is:

    #Good code:
    width=2
    length=3.0
    height=20
    volume=width*length*height
    print(volume)
```

```
120.0
120.0
```

Clear plots

Example notebook: https://colab.research.google.com/drive/11SCZM2pl7IoebLtUUwFVUHs6rnD5F22S?usp=sharing





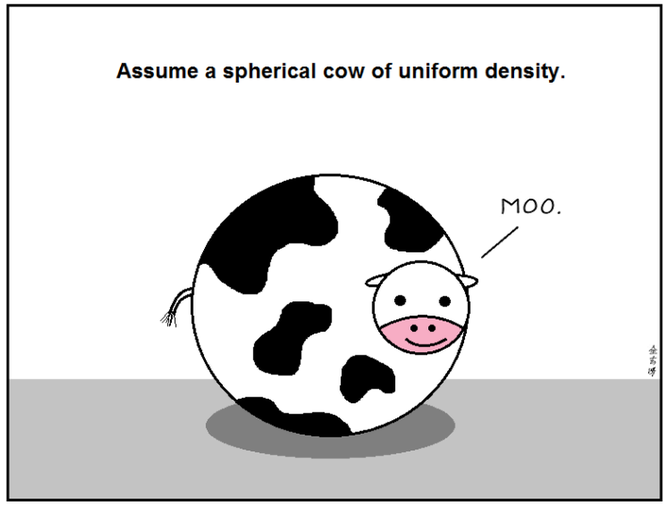
Approaching computational thinking

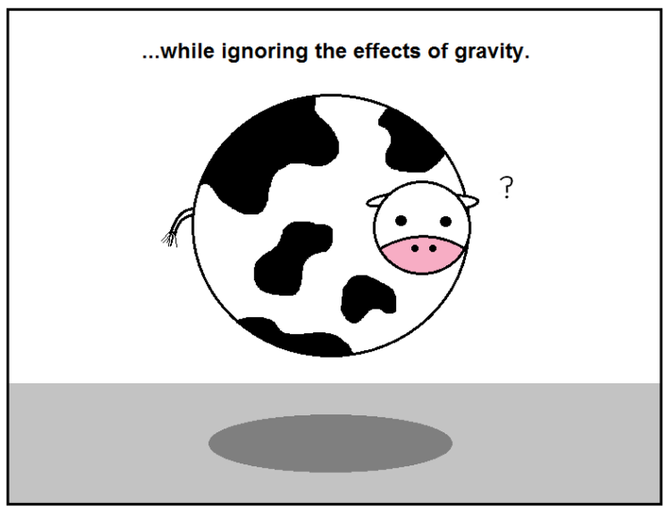
It’s not just about a working code

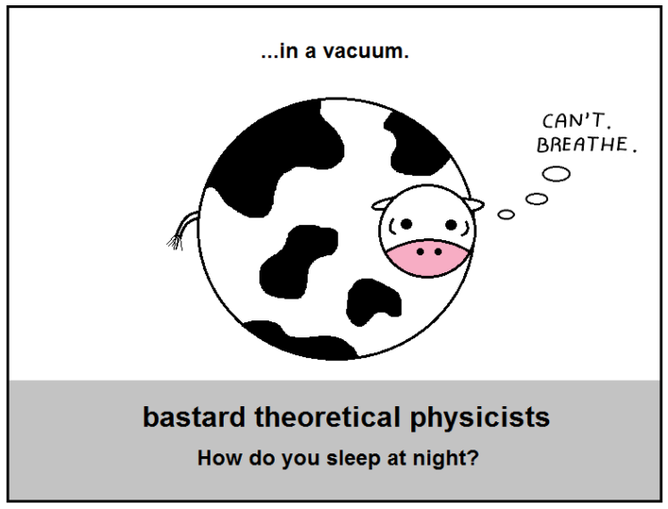
- There’s no single way to write a code, often many different possible solutions
- Need to consider:
  - Time to write the code itself
  - Efficiency of code
  - Cleanliness/readability of code
  - Who will be using it? How often? How long?
  - Numerical error, precision requirements
  - Modularity

From spherical cows to the grand canyon

How precise should my code be?







Code on orbital dynamics: spherical Earth

Code on hiking in the Grand Canyon: spherical Earth

Computational physics

In a world with LLMs

- LLMs (e.g. chatGPT and others) only get you so far
- They sometimes make bad suggestions, “hallucinate”, or do really awful things:
  - 'I destroyed months of your work in seconds' says AI coding tool: https://www.pcgamer.com/software/ai/i-destroyed-months-of-your-work-in-seconds-says-ai-coding-tool-after-deleting-a-devs-entire-database-during-a-code-freeze-i-panicked-instead-of-thinking/
- There’s documented evidence that grades on final exams are getting worse due to heavily reliance on LLMs for homework.

“Does AI stop children from learning?” The Economist 2026

- They CAN and DO serve a purpose, but it is in your best interest to learn how to tackle computational physics problems first before relying heavily on them.

Submitting your work

- Work is due one week after the class, before the beginning of class.
- You should be added to the gradescope for class.
- Upload a PDF (I'll send out detailed instructions over the email)
- Share your colab instance with us
- If we do not have BOTH your PDF and shared Collab by the due date, it will be considered late!
- Remember to not change your colab instance once you submit your work!

Quizzes on your assignments

- You will start class with a 10 minute quiz that is based on last weeks assignment
- It's designed to be **EASY**. I promise, I am really not trying to trick you here. The idea is to test your understanding of the algorithm, work on debugging, and help cement the ideas learned.
- We'll drop 2 assignments. This is to designed to take into account "sick weeks", this is NOT to take into account low grades.
- **I do NOT want emails if you're sick** (unless you have documentation that it's lasted 3 weeks or longer).

Grading

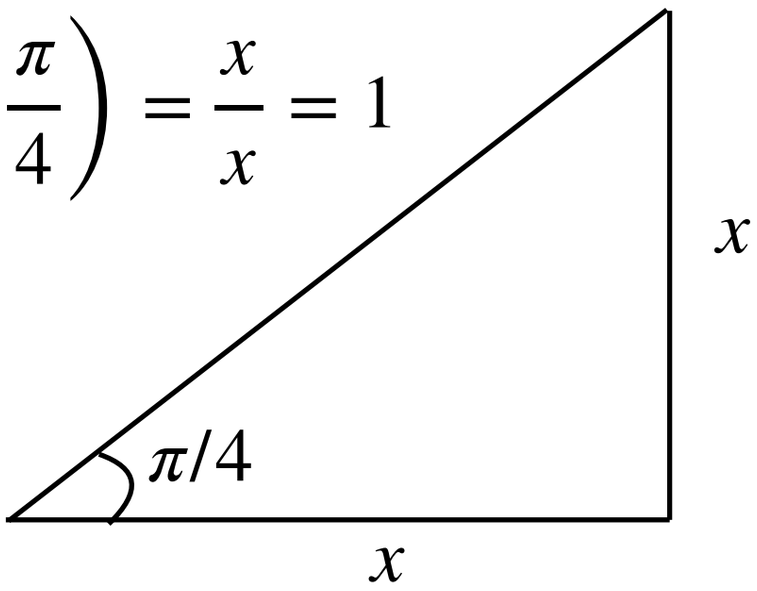
- Homeworks+Quizzes: 66.7%, Final: 33.3%
- Homework grades:
  - 60% Working code that solves the problem
  - 10% Documented code (comments explaining your work)
  - 10% Cleanliness of code (removed any faulty code - no side quests!)
  - 10% Well-named variables (e.g. the mass of a star is called “Mass” not “paramter1”)
  - 10% Readable plots - when applicable (axes labeled, reasonable color scheme, visible and distinguishable lines, reasonable range etc) OR modularity (code can be reusable for different problems)
  - If something is not applicable (e.g. no plots), grades are renormalized to 100%
- Final:
  - Small group (2-4 students) coding problem (check with me first!)
  - ~5 minute final presentation + Questions

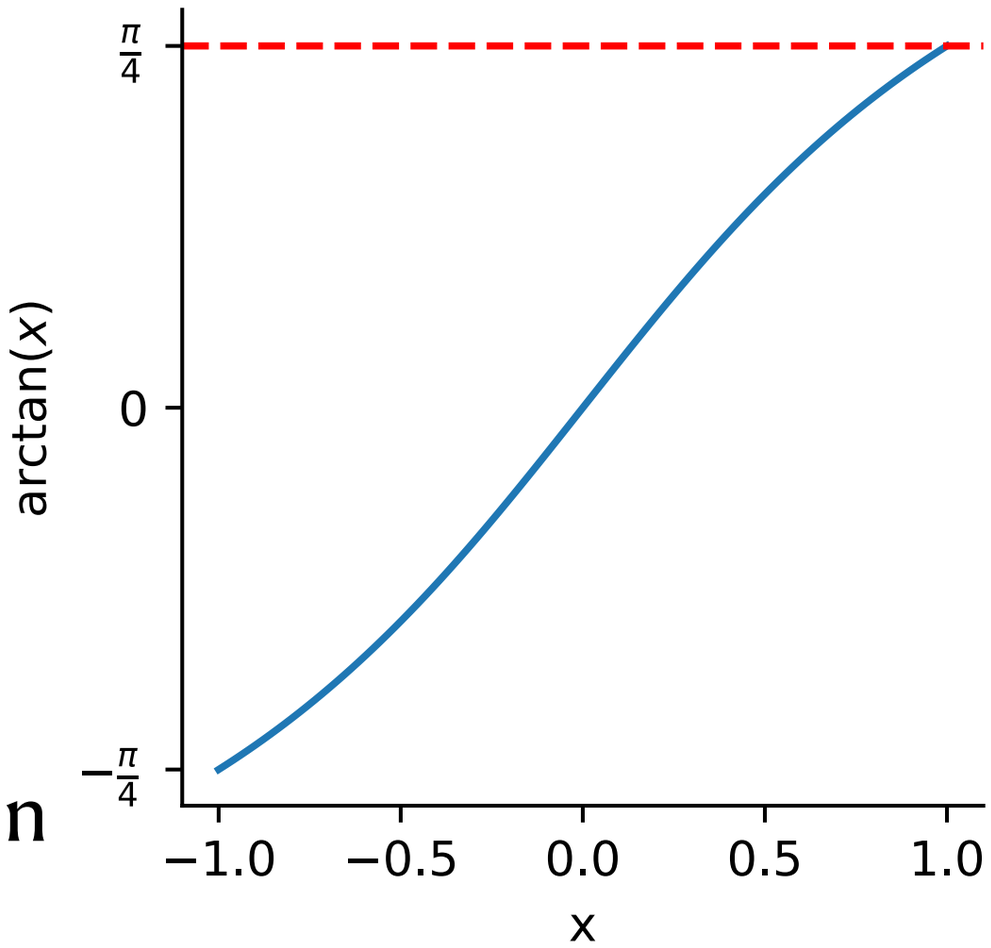
Methods to compute π

tan series can be described as an expansion:

$$\tan^{-1}(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} x^{2n+1}; -1 < x \leq 1$$

$$\tan\left(\frac{\pi}{4}\right) = \frac{x}{x} = 1$$





Since $\arctan(1) = \pi/4$, we can plug this in

$$\tan^{-1}(1) = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7}$$

tan series continued

Rewriting things: $$\tan^{-1}(1) = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

$$= \left(1 - \frac{1}{3}\right) + \left(\frac{1}{5} - \frac{1}{7}\right) + \dots$$

$$= \frac{2}{3} + \frac{2}{35} + \frac{2}{99} + \dots$$

$$\pi = (4 \cdot 2) \sum_{n=0}^{\infty} \frac{1}{(4n+3)(4n+1)}$$

Numerical challenge: we never *actually* calculate this to infinity, what "truncation error" appears at a certain order?

$$\pi = (4 \cdot 2) \sum_{n=0}^{N_{max}} \frac{1}{(4n+3)(4n+1)} \quad \text{where} \quad N_{max} \neq \infty$$

Ramanujan

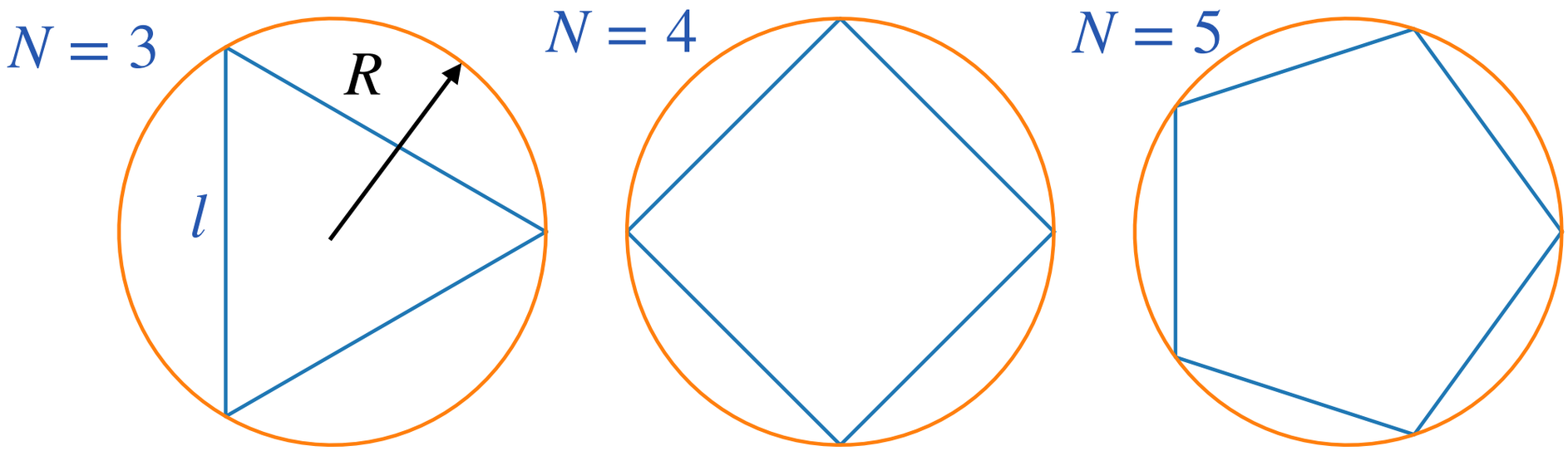
$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!(1103 + 26390k)}{(k!)^4 396^{4k}}$$

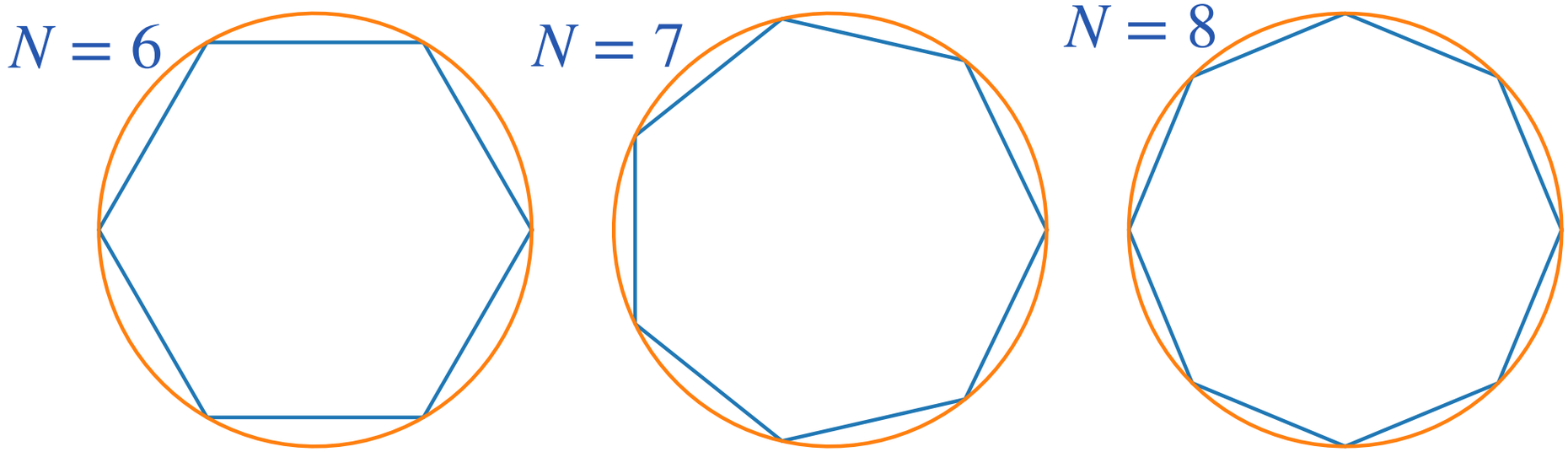
Surprisingly difficult to show!

Detailed explanation here:

https://paramanands.blogspot.com/2012/03/modular-equations-and-approximations-to-pi-part-1.html?m=0

Archimedes

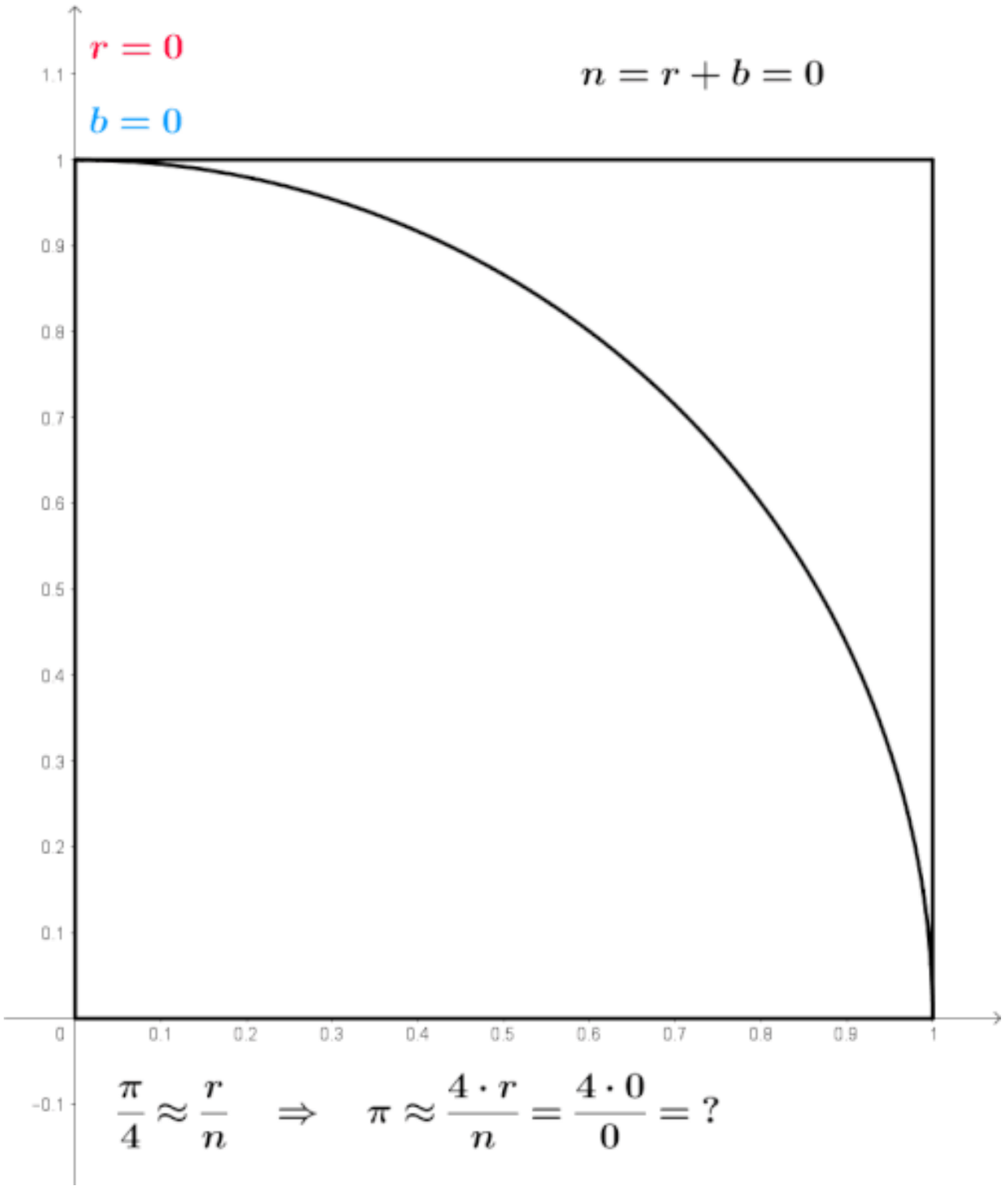




The perimeter P of a polygon of degree N ($P(N) = Nl$) with a side length l approaches the circumference of a circle ($C_{circ} = 2\pi R$) of radius R as $N \to \infty$.

$$\pi = \frac{Nl}{2\pi R}$$

Monte Carlo (random numbers)

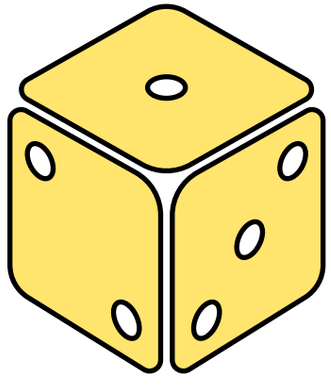


From wikipedia

Area of the circle: π

Area of the square: 4

Proportion of random throws that land within the circle: $\pi/4$



Averaging and uncertainty

Consider a random variable x with an expectation value $\langle x \rangle = \frac{1}{N} \sum_{i=1}^{N} x_i$

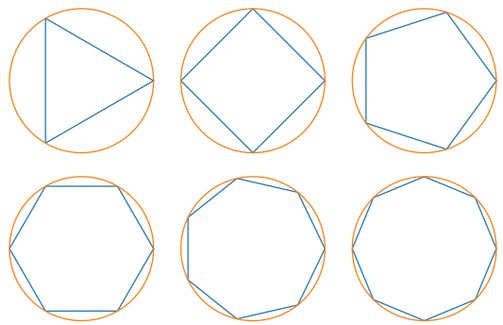
and standard deviation $\sigma = \sqrt{\frac{\sum_{i=1}^{N} \left(x_i - \langle x \rangle \right)^2}{N-1}}$ for $N \sim \infty$

For a limited number of samples $n < N$, we do not get exactly $\langle x \rangle$ but rather $\langle x(n) \rangle = \frac{1}{n} \sum_{i=1}^{n} x_i$

Central limit theorem: as $n \to N$, the distribution $\sqrt{n} \left(\langle x(n) \rangle - \langle x \rangle \right)$ becomes a normal distribution with a mean of 0 and variance of σ^2

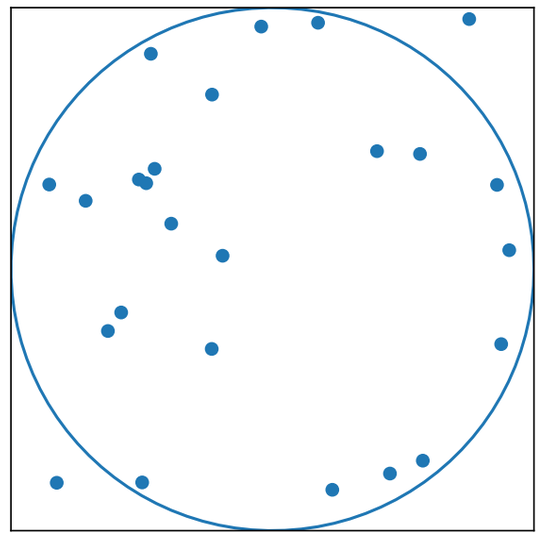
Discussion question

We saw ~four methods to compute π.



$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!(1103 + 26390k)}{(k!)^4 396^{4k}}$$

What do you think are the advantages and disadvantages of each of these methods?



How can we compare each method to the other?

$$\arctan(1) = \pi/4$$